

Guía de Laboratorio 4 — Derivada y Gradiente

El código Python y las visualizaciones detrás de la teoría

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán
Gestión: 2026

Carrera: Ingeniería Informática · UATF

La teoría de la derivada, las derivadas parciales, el gradiente y el descenso de gradiente ya la desarrollamos y estudiamos en clase. **En esta práctica no volvemos a demostrar fórmulas:** tomamos las derivadas que ya obtuvimos *a mano* y nos concentramos en un solo objetivo — **entender cómo el código de Python convierte esas fórmulas en gráficos**. Recorreremos, celda por celda, las cuatro visualizaciones del cuaderno: la recta tangente, la superficie con curvas de nivel, el campo del gradiente y la trayectoria del descenso de gradiente.

01 Objetivos de la práctica

Al terminar la guía, cada estudiante será capaz de:

- Configurar un entorno de graficación con **NumPy** y **Matplotlib**, y fijar un estilo global reutilizable.
- Traducir una función matemática a una **función de Python** y evaluarla de forma vectorizada sobre un arreglo de puntos.
- Explicar, línea por línea, cómo se dibuja una **recta tangente**, una **superficie 3D** con sus **curvas de nivel**, un **campo de vectores** (gradiente) y la **trayectoria** de un descenso de gradiente.
- Dominar las funciones clave de graficación: `linspace`, `meshgrid`, `plot`, `scatter`, `contour/contourf`, `plot_surface` y `quiver`.
- Modificar los parámetros del código (tasa de aprendizaje, punto inicial, densidad de la malla) y **predecir el efecto** sobre el gráfico resultante.

02 La caja de herramientas de graficación

Antes de leer las celdas, conviene tener a mano el vocabulario. Todas las visualizaciones se arman combinando unas pocas funciones. Esta es la referencia rápida que usaremos una y otra vez:

Función	Qué hace	Dónde la usamos
<code>np.linspace(a, b, n)</code>	Genera n puntos igualmente espaciados entre a y b. Es el dominio del gráfico.	En todas las curvas y mallas.

Función	Qué hace	Dónde la usamos
<code>np.meshgrid(x, y)</code>	Convierte dos ejes 1D en dos matrices 2D de coordenadas, para evaluar $f(x,y)$ en toda una cuadrícula.	Superficie, curvas de nivel, campo del gradiente.
<code>plt.plot(x, y)</code>	Dibuja una línea uniendo los puntos (x, y) .	Parábola, tangente, trayectoria.
<code>plt.scatter(x, y)</code>	Dibuja puntos sueltos (marcadores).	Punto de tangencia, mínimo.
<code>plt.contour / contourf</code>	Dibuja curvas de nivel (líneas) o bandas rellenas de igual altura.	Mapas topográficos de $f(x,y)$.
<code>ax.plot_surface(X,Y,Z)</code>	Dibuja la superficie 3D $z = f(x, y)$.	Vista 3D del "tazón".
<code>plt.quiver(X,Y,U,V)</code>	Dibuja una flecha (U, V) en cada punto (X, Y) : un campo de vectores.	Campo del gradiente ∇f .

03 Preparación del entorno

La primera celda importa las librerías, guarda los colores de la materia como constantes y fija un **estilo global** para que todas las figuras salgan con la misma apariencia sin repetir configuración.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # habilita la proyección 3D

# Paleta INF672 – "cuaderno científico"
INDIGO = "#4F46E5"
PURPURA = "#7E22CE"
CORAL = "#F43F5E"
TINTA = "#111827"

plt.rcParams["figure.figsize"] = (8, 5)
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3
plt.rcParams["font.size"] = 11
plt.rcParams["axes.edgecolor"] = "#9CA3AF"

print("Entorno listo ✓")

Out[1]: Entorno listo ✓
```

› Cómo se lee esta celda

- `import numpy as np` → NumPy nos da los **arreglos** (arrays) y la matemática vectorizada: operar sobre miles de números a la vez, sin bucles.
- `import matplotlib.pyplot as plt` → Matplotlib es la librería de dibujo. El apodo `plt` es la convención universal.

- `from mpl_toolkits.mplot3d import Axes3D` → no se usa por su nombre; se importa solo para *activar* la capacidad de dibujar en 3D (por eso el comentario). Sin esta línea, `projection="3d"` fallaría.
- Las constantes de color (INDIGO, CORAL...) guardan los códigos hexadecimales una sola vez para reutilizarlos en cada gráfico y mantener la identidad visual.
- `plt.rcParams[...]` → es el **diccionario de configuración global** de Matplotlib. Lo que fijamos aquí (tamaño de figura, rejilla activada al 30 % de opacidad, tamaño de letra, color del borde) se aplica automáticamente a *todas* las figuras siguientes.

CLAVE DE CÓDIGO

Configurar una sola vez, dibujar muchas veces. Poner el estilo en `rcParams` evita repetir `figsize`, `grid`, etc. en cada celda. Es la forma limpia de dar coherencia a todo un cuaderno.

04 Visualización 1 · La recta tangente

Recordatorio de una línea: ya derivamos a mano que para $f(x) = x^2$ la pendiente es $f'(x) = 2x$. El siguiente código **no** la recalcula: la recibe ya hecha y la usa para trazar la recta tangente en el punto $x_0 = 1.5$.

```
In [2]: f = lambda x: x**2          # función
fp = lambda x: 2*x                # derivada obtenida A MANO:  $f'(x) = 2x$ 

x0 = 1.5
x = np.linspace(-3, 3, 400)
tangente = f(x0) + fp(x0) * (x - x0)  # recta:  $y = f(x_0) + f'(x_0)(x - x_0)$ 

plt.plot(x, f(x), color=INDIGO, lw=2.5, label=r"$f(x)=x^2$")
plt.plot(x, tangente, color=CORAL, lw=2, ls="--",
         label=fr"tangente en  $x_0={x_0}$ ")
plt.scatter([x0], [f(x0)], color=PURPURA, s=70, zorder=5)
plt.axhline(0, color="#9CA3AF", lw=0.8); plt.axvline(0, color="#9CA3AF", lw=0.8)
plt.title(f"Pendiente de la tangente =  $f'({x_0}) = {fp(x_0)}$ ", color=TINTA)
plt.legend(); plt.show()
```

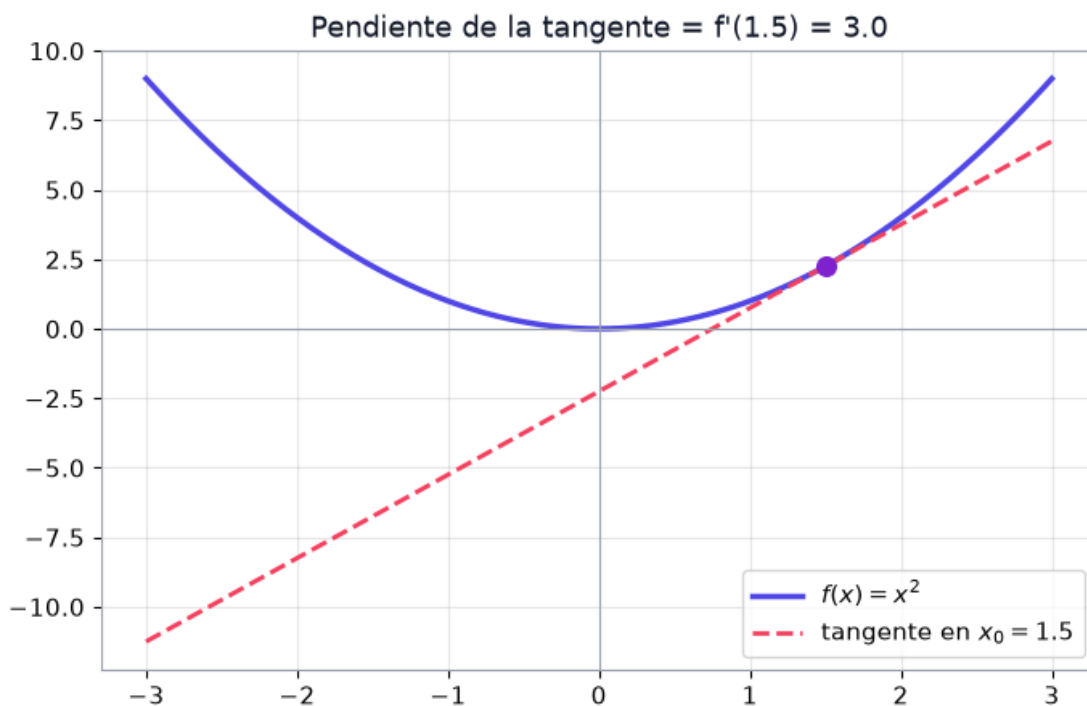


Figura 1 · Parábola $f(x)=x^2$ (índigo), su tangente en $x_0=1.5$ (coral punteada) y el punto de tangencia (púrpura).

› Construcción del gráfico, paso a paso

- `f = lambda x: x**2` y `fp = lambda x: 2*x` → dos **funciones anónimas**. Como `**` opera elemento a elemento, `f(x)` funciona igual con un número o con un arreglo completo.

- `x = np.linspace(-3, 3, 400)` → crea 400 valores de x entre -3 y 3 . Más puntos = curva más suave.
- `tangente = f(x0) + fp(x0)*(x - x0)` → la **ecuación punto-pendiente** $y = f(x_0) + f'(x_0)(x - x_0)$, escrita tal cual. Aquí es donde *usamos* la derivada: $fp(x_0)$ es la pendiente. La operación se aplica a los 400 puntos de golpe (vectorización).
- `plt.plot(x, f(x), color=INDIGO, lw=2.5, label=...)` → dibuja la parábola. `lw` es el grosor de línea; `label` es el texto para la leyenda. El prefijo `r"$...$"` activa notación matemática (LaTeX).
- `plt.plot(x, tangente, ls="--")` → la tangente en coral; `ls="--"` la hace punteada. El prefijo `fr"..."` combina *f-string* (inserta el valor de x_0) con notación matemática.
- `plt.scatter([x0], [f(x0)], s=70, zorder=5)` → marca el punto de tangencia. `s` es el tamaño del punto; `zorder=5` lo dibuja *encima* de las líneas.
- `plt.axhline(0) / plt.axvline(0)` → dibujan los ejes X e Y en gris.
- `plt.title(f"... {fp(x0)}")` → el *f-string* evalúa $fp(1.5)=3.0$ y lo escribe en el título.
- `plt.legend(); plt.show()` → muestra la caja de rótulos y renderiza la figura.

NOTA

Lectura del gráfico: la recta coral "besa" la parábola en un solo punto y tiene exactamente la inclinación 3.0 que da nuestra fórmula. Ese es el sentido geométrico de la derivada: la pendiente instantánea.

05 Visualización 2 · Superficie y curvas de nivel

Una función de dos variables, $f(x, y) = x^2 + xy + y^2$, ya no es una curva sino una **superficie**. La dibujamos de dos formas complementarias en una sola figura: la vista 3D (izquierda) y su “mapa topográfico” de curvas de nivel (derecha).

```
In [3]: f2 = lambda x, y: x**2 + x*y + y**2

x = np.linspace(-3, 3, 120)
y = np.linspace(-3, 3, 120)
X, Y = np.meshgrid(x, y)
Z = f2(X, Y)

fig = plt.figure(figsize=(12, 5))

ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.plot_surface(X, Y, Z, cmap="viridis", alpha=0.92)
ax1.set_title("Superficie f(x,y) = x² + xy + y²")
ax1.set_xlabel("x"); ax1.set_ylabel("y"); ax1.set_zlabel("f")

ax2 = fig.add_subplot(1, 2, 2)
cs = ax2.contourf(X, Y, Z, levels=20, cmap="viridis")
ax2.contour(X, Y, Z, levels=20, colors="white", linewidths=0.4)
ax2.set_title("Curvas de nivel"); ax2.set_xlabel("x"); ax2.set_ylabel("y")
ax2.set_aspect("equal")
fig.colorbar(cs, ax=ax2)
plt.tight_layout(); plt.show()
```

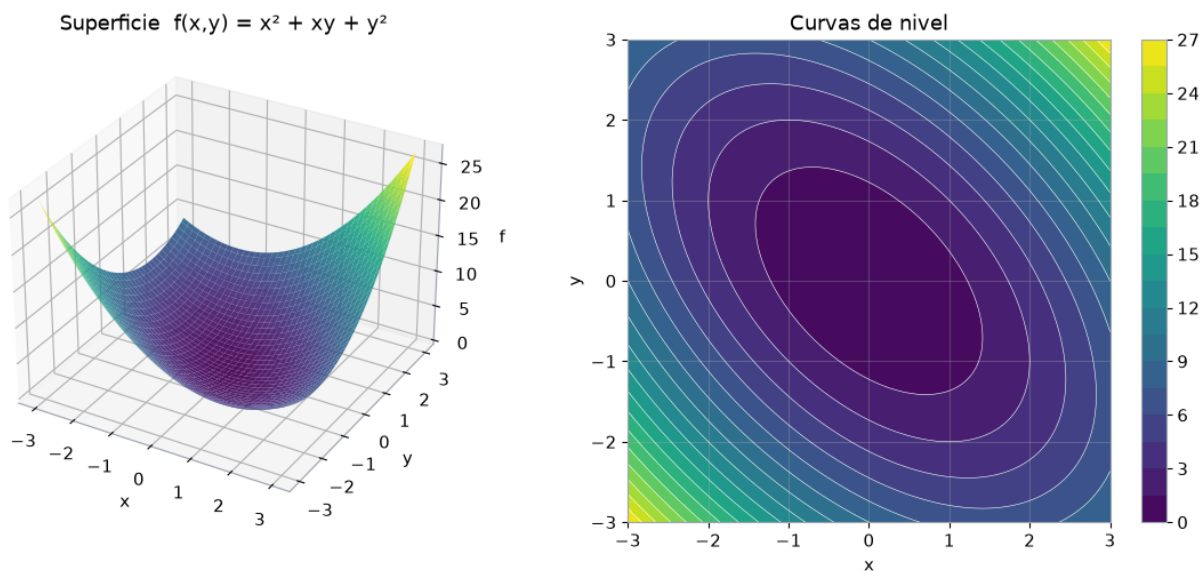


Figura 2 · Izq.: superficie 3D coloreada por altura. Der.: curvas de nivel (bandas de igual valor de f).

DEFINICIÓN

np.meshgrid — la idea clave. **linspace** nos da los ejes por separado (120 valores de x y 120 de y). **meshgrid** los combina en dos matrices 120×120 , x e Y , que contienen las coordenadas de **cada**

casilla de una cuadrícula. Así, $z = f_2(x, y)$ evalúa la función en los 14 400 puntos *de una sola vez*, sin ningún bucle. Sin meshgrid no podríamos dibujar superficies.

› Recorrido del código

- `fig = plt.figure(figsize=(12, 5))` → crea un lienzo ancho porque vamos a colocar **dos** gráficos lado a lado.
- `fig.add_subplot(1, 2, 1, projection="3d")` → "1 fila, 2 columnas, casilla 1", en modo 3D. `ax1` es el sub-eje izquierdo.
- `ax1.plot_surface(X, Y, Z, cmap="viridis", alpha=0.92)` → dibuja la superficie. `cmap` colorea según la altura (morado bajo → amarillo alto); `alpha` da una leve transparencia.
- `ax2 = fig.add_subplot(1, 2, 2)` → la casilla 2, en 2D, para el mapa de niveles.
- `ax2.contourf(..., levels=20)` → rellena 20 bandas de color según el valor de f (la "f" de `contourf` es de *filled*).
- `ax2.contour(..., colors="white")` → superpone las líneas blancas de nivel sobre las bandas rellenas.
- `ax2.set_aspect("equal")` → fuerza la misma escala en X e Y, para que las elipses no se vean deformadas.
- `fig.colorbar(cs, ax=ax2)` → añade la barra de color que traduce color ↔ valor de f .
- `plt.tight_layout()` → ajusta los márgenes automáticamente para que los dos gráficos no se solapen.

06 Visualización 3 · El campo del gradiente

El gradiente reúne las dos parciales en un vector: $\nabla f = (2x + y, x + 2y)$ (ya calculado a mano). Para verlo, dibujamos una flecha en cada punto: hacia dónde apunta y con qué fuerza crece la función.

```
In [4]: # Gradiente obtenido A MANO:  $\nabla f = (2x + y, x + 2y)$ 
def grad_f2(x, y):
    return 2*x + y, x + 2*y

# Malla dispersa para Las flechas
xg = np.linspace(-3, 3, 20)
yg = np.linspace(-3, 3, 20)
Xg, Yg = np.meshgrid(xg, yg)
U, V = grad_f2(Xg, Yg)

# Malla fina para Los contornos de fondo
xf = np.linspace(-3, 3, 200); yf = np.linspace(-3, 3, 200)
Xf, Yf = np.meshgrid(xf, yf); Zf = f2(Xf, Yf)

plt.contour(Xf, Yf, Zf, levels=15, cmap="viridis", alpha=0.6)
plt.quiver(Xg, Yg, U, V, color=CORAL, alpha=0.85)
plt.title("Campo del gradiente  $\nabla f$  (las flechas apuntan cuesta arriba)", color=TINTA)
plt.xlabel("x"); plt.ylabel("y"); plt.gca().set_aspect("equal")
plt.show()
```

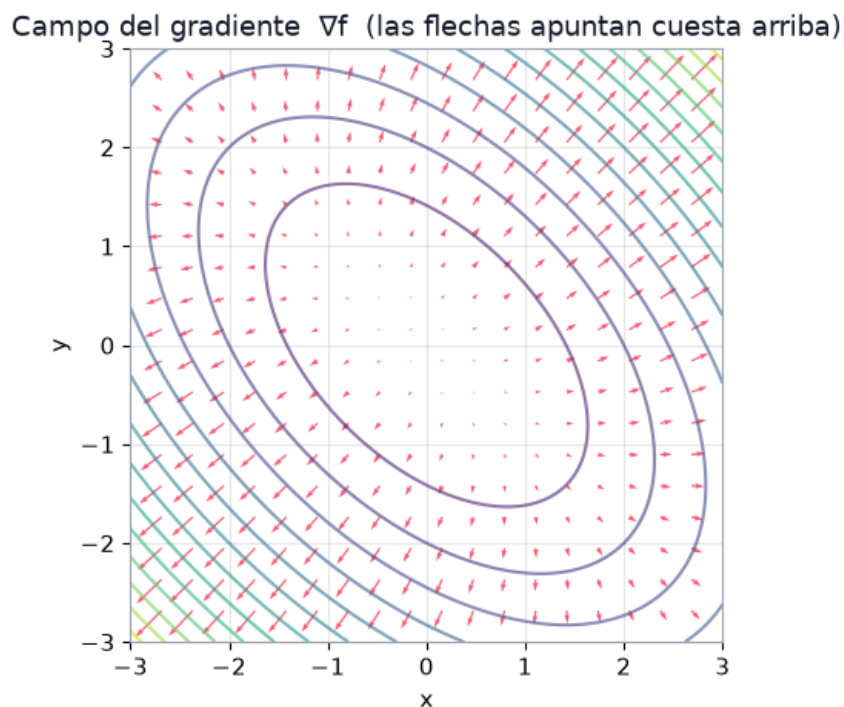


Figura 3 · Curvas de nivel de fondo y flechas del gradiente (coral). Cada flecha apunta “cuesta arriba” y es perpendicular a la curva de nivel.

› Por qué hay dos mallas

- `def grad_f2(x, y): return 2*x + y, x + 2*y` → una función que devuelve **dos valores** (una tupla): las dos componentes del gradiente. Es nuestra fórmula, escrita en Python.
- Malla dispersa 20×20 (X_g , Y_g) → pocas flechas, bien separadas. Si pusiéramos 200×200 flechas, se amontonarían en una mancha ilegible.
- `U, V = grad_f2(Xg, Yg)` → U y V son las componentes horizontal y vertical de cada flecha en cada punto de la malla dispersa.
- Malla fina 200×200 (X_f , Y_f , Z_f) → al contrario, las curvas de nivel del fondo sí necesitan muchos puntos para verse suaves.
- `plt.quiver(Xg, Yg, U, V, color=CORAL)` → el corazón del gráfico: coloca en cada base (X_g , Y_g) una flecha con dirección (U , V). Esa es la definición de **campo de vectores**.
- `plt.gca().set_aspect("equal")` → `gca()` toma el eje actual ("get current axes") para igualar la escala.

NOTA

Dos hechos que confirma el dibujo: (1) las flechas son **perpendiculares** a las curvas de nivel, y (2) apuntan hacia donde la función **crece**. Guarda esta imagen: en la siguiente sección iremos justo en dirección *contraria* para bajar hacia el mínimo.

07 Visualización 4 · Descenso de gradiente

Última pieza. Si ∇f apunta al máximo ascenso, entonces $-\nabla f$ apunta al descenso. El algoritmo da pasos pequeños en esa dirección hasta llegar al mínimo. Usamos el “tazón” $f(x, y) = x^2 + y^2$, con gradiente $\nabla f = (2x, 2y)$.

› Parte A · El bucle que calcula la trayectoria

```
In [5]: f3      = lambda x, y: x**2 + y**2
grad_f3 = lambda x, y: np.array([2*x, 2*y])    #  $\nabla f = (2x, 2y)$ , derivado a mano

punto = np.array([4.0, 4.0])    #  $x_0$ 
alpha = 0.1                    # tasa de aprendizaje
trayectoria = [punto.copy()]

for _ in range(25):
    punto = punto - alpha * grad_f3(*punto)    #  $x_{k+1} = x_k - \alpha \nabla f(x_k)$ 
    trayectoria.append(punto.copy())

trayectoria = np.array(trayectoria)
print("Primeros puntos:")
for i, p in enumerate(trayectoria[:3]):
    print(f"  x_{i} = ({p[0]:.3f}, {p[1]:.3f})")
print(f" ... \n  x_25 = ({trayectoria[-1,0]:.4f}, {trayectoria[-1,1]:.4f})")

Out[5]: Primeros puntos:
  x_0 = (4.000, 4.000)
  x_1 = (3.200, 3.200)
  x_2 = (2.560, 2.560)
  ...
  x_25 = (0.0151, 0.0151)
```

- `grad_f3 = lambda x, y: np.array([2*x, 2*y])` → devuelve un `np.array`, no una tupla, para poder hacer aritmética de vectores (`punto - alpha*grad`) directamente.
- `punto = np.array([4.0, 4.0])` → el punto de partida x_0 . Los `.0` fuerzan números decimales.
- `trayectoria = [punto.copy()]` → guardamos el recorrido. El `.copy()` es importante: sin él guardaríamos una **referencia** que cambiaría en cada paso, y todos los registros quedarían iguales.
- `punto = punto - alpha * grad_f3(*punto)` → la **regla de actualización** $x_{k+1} = x_k - \alpha \nabla f(x_k)$, escrita en una línea. El `*punto` “desempaqueta” el arreglo en los argumentos x, y .
- `trayectoria.append(punto.copy())` → agrega la nueva posición al historial.
- Los f-strings con `:.3f` / `:.4f` → formatean los números a 3 o 4 decimales. La salida confirma que $x_0=(4,4)$ desciende hasta casi $(0,0)$.

NOTA

Compara con la cuenta a mano: $x_1 = (4,4) - 0.1 \cdot (8,8) = (3.2, 3.2)$, y $x_2 = (2.56, 2.56)$. El bucle hace exactamente esa resta, 25 veces. El código no “sabe” cálculo: solo repite nuestra fórmula.

▸ Parte B · Dibujar la trayectoria sobre el mapa

```
In [6]: xf = np.linspace(-5, 5, 200); yf = np.linspace(-5, 5, 200)
        Xf, Yf = np.meshgrid(xf, yf); Zf = f3(Xf, Yf)

        plt.contour(Xf, Yf, Zf, levels=20, cmap="viridis", alpha=0.6)
        plt.plot(trayectoria[:, 0], trayectoria[:, 1], "o-", color=CORAL, ms=4, lw=1.5,
                  label="trayectoria")
        plt.scatter([0], [0], marker="*", s=260, color=PURPURA, zorder=5, label="mínimo")
        plt.title("Descenso de gradiente: bajando hacia el mínimo", color=TINTA)
        plt.xlabel("x"); plt.ylabel("y"); plt.legend(); plt.gca().set_aspect("equal")
        plt.show()
```

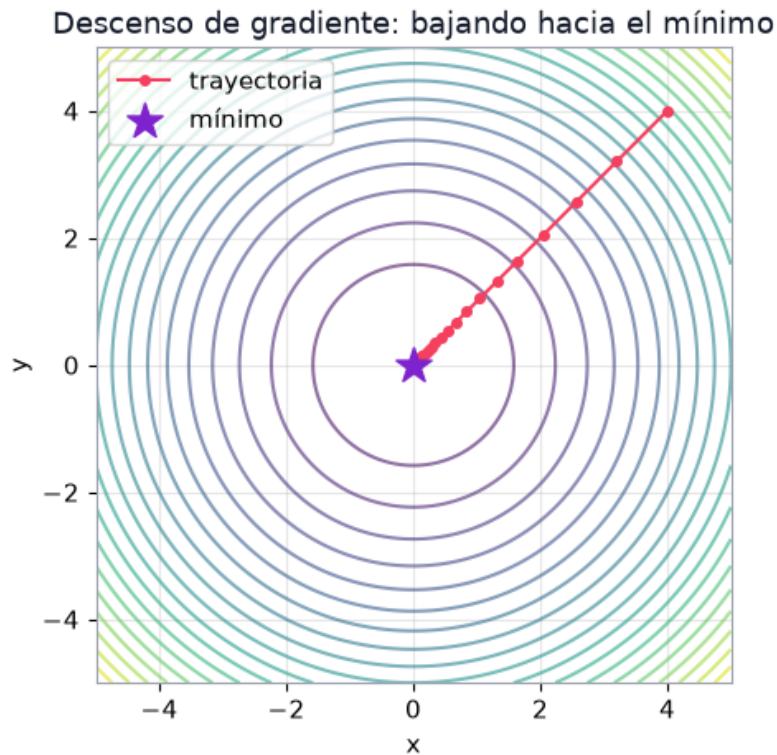


Figura 4 · Trayectoria del descenso (coral) sobre las curvas de nivel, descendiendo hasta el mínimo (estrella púrpura) en $(0, 0)$.

- `plt.contour(..., levels=20, alpha=0.6)` → dibuja el "tazón" como curvas de nivel de fondo, semitransparentes.
- `trayectoria[:, 0]` y `trayectoria[:, 1]` → rebanado de NumPy: `[:, 0]` toma la columna de las x ; `[:, 1]`, la de las y .
- `plt.plot(..., "o-", ms=4, lw=1.5)` → el formato "o-" pinta un marcador circular en cada paso *unido* por línea. `ms` es el tamaño del marcador.
- `plt.scatter([0],[0], marker="*", s=260)` → una estrella grande marca el mínimo conocido en $(0, 0)$.

- **Observa el espaciado:** los puntos están **más separados al inicio** (gradiente grande, pasos largos) y **se juntan cerca del mínimo** (gradiente pequeño, pasos cortos). El algoritmo “frena solo”.

CUIDADO

La tasa de aprendizaje α es delicada.

Si α es **demasiado grande**, los pasos se pasan de largo y el método puede **divergir** (oscila o se dispara).

Si α es **demasiado pequeña**, converge **muy lento**.

08 Entregables

Reproduce el cuaderno y entrega un .ipynb (o un PDF exportado) que incluya, además de las cuatro figuras originales ejecutadas, lo siguiente:

- **E1.** Resultados de los cuatro ejercicios guiados, cada uno con su gráfico y **una o dos frases** que expliquen lo observado.
- **E2.** Repite la Visualización 1 para una función distinta, $g(x) = x^3 - 3x$, con su derivada $g'(x) = 3x^2 - 3$ (derívala a mano) y dibuja la tangente en $x_0 = 1$.
- **E3.** En el descenso de gradiente, cambia el punto inicial a $(-4, 3)$ y comenta cómo cambia la trayectoria hacia el mínimo.
- **E4.** Escribe en una celda Markdown tu respuesta, con tus propias palabras, a la pregunta del cuaderno: *¿cómo sabe un modelo hacia qué lado mover cada parámetro para reducir el error?*

09 Rúbrica de evaluación

Criterio	Descripción	Puntos
Entorno y ejecución	El cuaderno corre completo sin errores y reproduce las cuatro figuras.	20
Comprensión del código	Explica correctamente las funciones clave (linspace, meshgrid, plot, contour, quiver).	25
Ejercicios guiados	Resuelve los 4 ejercicios con gráfico y observación escrita.	25
Entregables E2–E3	Nueva tangente y nuevo punto inicial, con derivadas a mano.	20
Reflexión final (E4)	Respuesta clara que conecta gradiente, descenso y entrenamiento.	10

Total: 100 puntos